

Buy Local Bonds

From Case Brief to Release 1 Backlog

Epics, user stories, refinement, classification, prioritisation, estimation and baselining — learner assessment

Course context: Agile Project Management — Product Definition; Core: Requirements — Requirements Defined, Elicitation Skills, Finalizing Requirements, Requirements Management.

Case: Meridian Financial / Buy Local Bonds (fictional; figures illustrative).

Contents: Nine assessed steps, each with instructions, a template and the expected output.

Table of contents

Learner Assessment Handout	2
Learning objectives	2
Logistics	2
Overview of the nine steps	2
Step 1 — Harvest needs from the case brief (20 min)	4
Step 2 — Write epics (25 min)	5
Step 3 — Break epics into user stories (30 min)	6
Step 4 — Fine-tune stories: INVEST, Standard of Quality, acceptance criteria (40 min)	7
Step 5 — Classify every item (25 min)	9
Step 6 — Select a prioritisation strategy (20 min)	11
Step 7 — Prioritise the backlog (25 min)	12
Step 8 — Estimate in story points (25 min)	13
Step 9 — Baseline Release 1 (30 min)	14

Learner Assessment Handout

Exercise: From Case Brief to Release 1 Backlog — Buy Local Bonds (BLB)

You are the requirements team for Buy Local Bonds. Meridian Financial wants a self-service platform that lets retail customers find municipal bonds funding projects near where they live and buy them without a broker. Your job is to turn the case brief into a prioritised, estimated, quality-checked Release 1 product backlog. Nine steps take you from raw needs to a baselined backlog; each step has a deliverable that is assessed.

Learning objectives

By the end of this exercise you will be able to:

- Write epics and user stories in the "As a ... I want ... so that ..." form that trace back to a business need.
- Refine stories against the INVEST guideline and the Standard of Quality (complete, consistent, unambiguous, well structured, correct, feasible, verifiable, traceable, necessary, prioritised, modifiable).
- Classify backlog items by type (BR / UR / SR) and by category (Assumption, Business Rule, Constraint, Dependency, Functional, Non-Functional).
- Select and justify a prioritisation strategy, apply it, and produce an ordered backlog.
- Estimate stories in relative story points and identify where a spike is needed.
- Baseline a Release 1 backlog with traceability from business need to acceptance criteria.

Logistics

Item	Detail
Team format	Teams of 4–5. One person acts as Product Owner (owns value and priority calls); the rest are analysts/developers. Rotate the PO role at Step 6.
Inputs	Buy Local Bonds case brief; course chapters: Product Definition (user stories, epics, themes, INVEST, backlog), Requirements Defined (types and categories), Elicitation Skills (Standard of Quality, SMART/testable), Finalizing Requirements (prioritisation), Requirements Management (Release 1 baseline).
Materials	Index cards or sticky notes, flip chart, the templates in this handout (or a shared spreadsheet with one tab per step), planning-poker cards or a 1-2-3-5-8-13-20 deck.
Total time	Approximately 4 hours across nine steps (timings per step below). Can be split over two sessions: Steps 1–5, then Steps 6–9.
Submission	One consolidated workbook or document per team containing the nine deliverables, plus the peer-review sheet from Step 4.
Ground rule	Where the case brief is silent (budget, launch date, minimum purchase, inventory), do not invent facts silently. Record what you assumed as an Assumption with a status and an owner who would confirm it.

Overview of the nine steps

Step	Activity	Deliverable	Time
1	Harvest needs from the case brief	Needs Register (business needs, user goals, known facts, open questions)	20 min
2	Write epics	Epic Register (5–7 epics, each traced to a need)	25 min
3	Break epics into user stories	Story deck (12+ stories from three epics)	30 min
4	Fine-tune stories: INVEST + Standard of Quality + acceptance criteria	Refined story cards with INVEST scorecard and peer-review sheet	40 min

Step	Activity	Deliverable	Time
5	Classify every item	Classification table (BR/UR/SR and A/B/C/D/FR/NF) and A-B-C-D Register	25 min
6	Select a prioritisation strategy	Strategy decision record with comparison and justification	20 min
7	Prioritise the backlog	Ordered backlog with MoSCoW (or chosen scheme) and value/risk scores	25 min
8	Estimate in story points	Estimated backlog, reference story, spikes identified	25 min
9	Baseline Release 1	Release 1 backlog with traceability matrix and final quality review	30 min

Step 1 — Harvest needs from the case brief (20 min)

Purpose

Stories that do not trace to a business need are gold plating. Before writing anything in story form, pull out what Meridian actually wants, what customers must be able to do, what is fixed, and what nobody knows yet.

Instructions

1. Read the case brief individually (5 min). Highlight every sentence that states a goal, a user need, a fixed fact, or an open question.
2. As a team, build the Needs Register below. Give every business need an ID (BN1, BN2 ...) so later steps can trace to it.
3. Record the four "still open" items and anything else you cannot find in the brief as Open Questions, with the stakeholder you would ask.
4. Do not write stories yet. This step is about needs, not solutions.

Template: Needs Register

ID	Business need / user goal (verbatim or paraphrased)	Source in brief	Type (Business goal / User goal / Known fact / Open question)	Who would confirm?

Expected output

- 4–6 business needs, including the stated strategy ("purchase bonds online without a broker") and the four "why Meridian is interested" points.
- 3–5 user goals derived from "The Idea" (find bonds near me, see what a bond pays for, see rating/yield/maturity, buy directly).
- Known facts captured as facts, not needs: existing customers first, phone and desktop, regulatory obligations, third-party market data.
- At least the four open questions from the brief, each with a named stakeholder role who could answer it.

Step 2 — Write epics (25 min)

Purpose

Epics are large stories that group related work. Written well, they give the backlog a shape that business stakeholders recognise and that every later story can hang from.

Instructions

1. Write 5–7 epics in the form "As a [type of user] I want [capability] so that [benefit]". Each epic should cover a whole area of need from Step 1, not a single screen.
2. Trace each epic to one or more Needs Register IDs. If an epic traces to nothing, either it is not an epic or you missed a need.
3. Check coverage: every business need in the register should be covered by at least one epic. If not, add one or record why it is out of scope.
4. Use different personas where they matter. "Customer" is not the only user; consider compliance, marketing, operations.
5. Flag anything you were tempted to write as an epic that is really a constraint or a quality attribute (for example, "works on a phone"). Park it for Step 5.

Template: Epic Register

Epic ID	Epic name	As a ... I want ... so that ...	Traces to (BN IDs)	Persona

Expected output

- 5–7 epics, each a complete three-part story statement with a real benefit clause (not "so that I can use the feature").
- A coverage check showing each BN ID mapped to at least one epic.
- At least one non-customer persona (for example a compliance officer).
- A "parked" list of constraint-like items that were not written as epics.

Step 3 — Break epics into user stories (30 min)

Purpose

A story describes one thing the system needs to do, from the user's point of view, with enough information to estimate. This step produces the raw deck; Step 4 polishes it.

Instructions

1. Pick three epics: the one with the highest business value, one that is regulatory, and one of your choice.
2. Individually, write as many stories as you can for those epics on index cards (10 min). One story per card. Use the standard form.
3. As a team, merge the cards: remove duplicates, split anything that is really two stories, and give each survivor an ID (US1.1, US1.2 ... under Epic 1).
4. Aim for 12–16 stories in total. Make them as small as you can while still delivering something a user would notice.
5. Do not write acceptance criteria yet, but jot down any questions a developer would ask on the back of the card.

Template: Story card

Field	Content
Story ID	US<epic>.<n>
Title	Short verb phrase, e.g. "Search by ZIP code"
Story	As a ... I want ... so that ...
Parent epic	E<n>
Developer questions	What would a developer need to ask before starting?

Expected output

- 12–16 stories across three epics, each with an ID, title, three-part statement and parent epic.
- Every story describes exactly one capability. "Search and buy" is two stories.
- Each story has at least one developer question recorded — this feeds acceptance criteria in Step 4.

Step 4 — Fine-tune stories: INVEST, Standard of Quality, acceptance criteria (40 min)

Purpose

A card is a reminder to have a conversation, but it still has to be testable and estimable. This step turns rough cards into stories a developer could start on and a tester could finish.

Instructions

1. Run every story through the INVEST scorecard (Independent, Negotiable, Valuable, Estimable, Small, Testable). Score each letter Pass / Weak / Fail and note why.
2. Rewrite any story with a Fail, or with two or more Weak. Keep the original wording on the card so the improvement is visible.
3. Write 2–4 acceptance criteria per story in Given / When / Then form. Each criterion must name the input, the action and the expected result in terms of data a tester can check.
4. Apply the Standard of Quality checklist to the whole set: unambiguous words only (no "fast", "easy", "appropriate", "etc."), consistent actor names, one capability per statement, no unresolved TBDs without an owner.
5. Swap decks with another team for 10 minutes. Review three of their stories using the peer-review sheet and hand it back. Incorporate feedback you agree with; record why you rejected the rest.

Template: INVEST scorecard

Story ID	I	N	V	E	S	T	Issue found	Action (keep / rewrite / split / merge)

Template: Refined story card

Field	Content
Story ID / Title	
Original wording	As written in Step 3
Refined wording	As a ... I want ... so that ...
Acceptance criteria	AC1: Given ... When ... Then ... AC2: ... AC3: ...
Quality notes	Which Standard-of-Quality attributes were fixed and how
Traces to	Epic ID → BN IDs

Template: Peer-review sheet

Reviewed story	Weakest INVEST letter	Ambiguous word(s) found	Could you write a test from the ACs? (Y/N)	Suggested fix

Reviewed story	Weakest INVEST letter	Ambiguous word(s) found	Could you write a test from the ACs? (Y/N)	Suggested fix

Expected output

- An INVEST scorecard covering every story, with at least three stories rewritten, split or merged, and the before/after visible.
- 2–4 Given/When/Then acceptance criteria per story, each checkable with known input and expected output.
- No vague quality words remaining; any quality attribute ("fast") has been turned into a measurable criterion or moved to the non-functional list for Step 5.
- A completed peer-review sheet and a short note on which feedback was adopted.

Step 5 — Classify every item (25 min)

Purpose

User stories are functional. Everything else the project must respect — assumptions, business rules, constraints, dependencies, quality attributes — still has to be captured, tracked and, in many cases, monitored as a risk. Classification also tells you who the audience of each statement is.

Instructions

1. For every story, assign a type: BR (business requirement — organisational objective), UR (user requirement — a user goal or expectation) or SR (system requirement — a capability the system must provide, written precisely). Most well-formed stories are UR; some, written from a compliance or system perspective, are SR.
2. For every story, assign one or more categories: A (Assumption), B (Business Rule), C (Constraint), D (Dependency), FR (Functional), NF (Non-Functional). An item can carry more than one.
3. Build the A-B-C-D-NF Register: go back through the case brief, your Step 1 open questions and the "parked" items from Step 2, and record every assumption, business rule, constraint, dependency and non-functional requirement as a separate numbered item. Give assumptions and dependencies a status and an owner — they are risks until confirmed.
4. Link each register item to the stories it affects.

Template: Classification table

ID	Statement (short)	Type BR/UR/SR	Category A/B/C/D/FR/NF	Why (one line)

Template: A-B-C-D-NF Register

ID	Category	Statement	Status (Confirmed / Unconfirmed / Open)	Owner / who confirms	Affects stories

Expected output

- Every story typed and categorised with a one-line reason.

- A register of at least 3 assumptions, 3 business rules, 3 constraints, 3 dependencies and 3 non-functional requirements, each with a unique ID.
- Every assumption and dependency carries a status and an owner.
- At least one item shown with two categories (for example a constraint that is also a dependency).

Step 6 — Select a prioritisation strategy (20 min)

Purpose

Prioritisation methods are project dependent. Before ranking anything, decide how you will rank, and be able to defend it to the Product Owner, the SOX compliance committee and the brokerage heads who dislike the whole idea.

Instructions

1. Rotate the Product Owner role.
2. Compare at least four techniques from the course: High/Medium/Low, Essential/Conditional/Optional, MoSCoW, Kano, plus one of the researched methods (100-point, Wieggers, relative weighting, theme scoring, stacked ranking).
3. Score each technique against what matters for BLB Release 1: handles regulatory "non-negotiables"; works when budget and date are unknown; is driven by the customer/PO; supports a clear "not this release" boundary; is quick enough for a workshop; distinguishes between items of equal priority.
4. Choose one primary technique and, if needed, one secondary technique to order items within a band. Write a decision record.

Template: Strategy comparison

Technique	Strength for BLB	Weakness for BLB	Fit score (1–5)

Template: Decision record

Field	Content
Decision	Primary technique ... ; secondary technique ...
Basis of prioritisation	Which of the Chapter 3 elements drive it: business value, business risk, technical risk, feasibility, time, budget, regulatory/policy compliance
Why not the alternatives	One line each
Who decides ties	Product Owner, with named escalation
Time-box	What "this release" means for the MoSCoW/Kano judgement

Expected output

- A comparison of four or more techniques with BLB-specific strengths and weaknesses (not textbook definitions copied out).
- A decision record naming the chosen technique(s), the prioritisation elements that drive it, and why the alternatives were rejected.

Step 7 — Prioritise the backlog (25 min)

Instructions

1. Apply the chosen technique to every story and to every register item that must be delivered (constraints and business rules are usually Must by definition; assumptions and dependencies are tracked, not prioritised).
2. Score business value (H/M/L) and risk (H/M/L, the higher of business and technical risk) for each story. Use these to order items within a MoSCoW band or Kano class.
3. Apply brutal prioritisation: challenge any item a team member insists is Must. Ask "what is the worst thing that happens if this is not in Release 1?" If the answer is "nothing measurable", it is not a Must.
4. Check that Must items alone form a usable product (a customer can find, understand, lawfully buy and get confirmation of a bond). If not, something is mislabelled.
5. Record every priority disagreement and how the PO resolved it.

Template: Prioritised backlog

Rank	ID	Title	Value H/M/L	Risk H/M/L	MoSCoW (or class)	Rationale

Expected output

- Every story ranked, with value, risk and priority band populated.
- No more than about 50% of stories in Must; at least one item explicitly Won't (this release).
- A written check that the Must set is a coherent minimum product.
- A log of at least two disagreements and their resolution.

Step 8 — Estimate in story points (25 min)

Instructions

1. Agree a reference story: the smallest well-understood story in the deck. Give it 2 points.
2. Estimate each story relative to the reference using a modified Fibonacci scale (1, 2, 3, 5, 8, 13, 20). Points reflect effort, complexity and uncertainty together, not hours.
3. Use planning poker: everyone reveals at once; the highest and lowest explain; re-vote until within one card. Record the final value and one line of rationale.
4. Any story estimated at 13 or more must be split or, if the uncertainty is technical (for example the third-party market data feed), replaced by a time-boxed spike plus a follow-on story.
5. Total the Must points. Assuming an illustrative velocity of 20 points per two-week sprint, state how many sprints Release 1 needs. Do not present this as a commitment; it is a planning signal.

Template: Estimated backlog

ID	Title	Points	Main driver of size (effort / complexity / uncertainty)	Split or spike?

Expected output

- Reference story named; every story has a Fibonacci value and a rationale.
- No story larger than 8 remains unsplit; at least one spike is identified with a time-box and the question it answers.
- Must-point total and an indicative sprint count with the velocity assumption stated.

Step 9 — Baseline Release 1 (30 min)

Instructions

1. Assemble the Release 1 backlog: all Must stories in rank order, plus the Should stories that fit the indicative capacity, plus the register items that apply to Release 1.
2. Build the traceability matrix: Business need → Epic → Story → Acceptance criteria → (placeholder) test case ID. Every Release 1 story must trace upward to a BN and downward to at least one AC.
3. Run the Verification Checklist over the baseline (complete, consistent, unambiguous, well structured, correct, feasible, verifiable, traceable, necessary, prioritised, modifiable). Record a Pass / Fix for each attribute and what you fixed.
4. Organise the backlog for its audience: state whether you grouped by usage, by audience or by quality, and why.
5. Name the document you will baseline this in, and how a change to it would be processed.
6. Prepare a 60-second readout: what is in Release 1, what is deliberately out, the biggest assumption, the biggest dependency.

Template: Traceability matrix

BN	Epic	Story	Priority	Points	AC IDs	Test case ID	Register items

Template: Verification checklist

Attribute	Pass / Fix	Evidence or what was fixed

Expected output

- A Release 1 backlog of roughly 8–12 stories with ranks, points and cumulative points, plus applicable register items.
- A complete traceability matrix with no orphan stories (no BN) and no untestable stories (no AC).
- A verification checklist with at least two "Fix" entries showing what changed as a result.
- Organisation approach and baseline document named; change process described in two or three sentences.
- The 60-second readout delivered to the class.